

Technical Test - PDF Search API

Objective

Imagine you are building a research tool for analysts working with large collections of public documents, such as parliamentaryO proceedings, public hearings, official reports...etc.

The goal of the system is to help users quickly identify the most relevant passages and documents related to a topic without manually reviewing every document.

Build a small backend system composed of two parts:

1. A Python command-line ingestion program that processes PDF documents.
2. A FastAPI application that serves search results through an API.

You will be given access to a [Google Drive folder](#) containing a few PDF documents. Your ingestion program should process the PDFs after they have been downloaded locally. The local folder path containing the PDFs must be provided as a command-line argument.

The goal of this exercise is not to build a perfect production-ready system, but to understand how you approach a problem, break it into components, structure your code, make technical decisions and execute a working solution.

We are primarily interested in your engineering approach, code quality, project structure and ability to explain your technical choices and trade-offs. We are less interested in advanced retrieval techniques or achieving the highest possible search accuracy.

This exercise should take approximately **2 to 4 hours** to complete. We value your time and do not expect you to spend significantly longer than that.

You are welcome to use AI-assisted development tools such as ChatGPT, Claude, Cursor, GitHub Copilot or similar tools. In fact, we encourage it. What matters to us is your ability to leverage available tools effectively, understand the solution you build and explain the decisions you made during the review discussion.

Expected components

1. Ingestion Program

Create a **Python command-line program** responsible for processing the documents.

It should:

- Read all PDF files from the input folder passed as a command-line argument.
- Extract text from each PDF.
- Split the extracted text into meaningful chunks.
- Generate embeddings for each chunk using a local open-source model runnable on CPU. As the documents are in French, you may use a multilingual embedding model such as *sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2*.
- Store the vectors in a local vector index such as FAISS, and persist the index and metadata to disk so the API can load them after ingestion.
- Store the metadata associated with each chunk, including at minimum the document name, page number, chunk index and original text content.

Notes:

1. The embedding models and vector databases mentioned above are only examples. You are free to use any model, library or local vector database you consider appropriate, as long as the solution can be run locally without requiring a paid external API.
2. The PDF documents are real-world documents and the extracted text may contain formatting issues, duplicated content, headers, footers or other artefacts commonly encountered during PDF processing. Do not worry too much about perfectly cleaning the extracted text. We are more interested in your overall approach, engineering decisions and solution design than in achieving perfect extraction quality.
3. We encourage you to make pragmatic decisions and clearly document any assumptions or trade-offs in the README.

2. API Server

Create a **FastAPI** application responsible for serving search results from the indexed documents.

It should expose at least:

POST /search

Accepts a user query, performs a similarity search against the indexed document chunks and returns the most relevant results along with their associated metadata.

The response should include sufficient information for a client application to understand where the result came from, including at minimum:

- document name
- page number
- chunk index
- similarity score (if available)
- chunk content

Example request:

```
{
  "query": "Quelle est la position du document sur les politiques publiques ?",
  "top_k": 5
}
```

Example response:

```
{
  "query": "Quelle est la position du document sur les politiques publiques ?",
  "results": [
    {
      "document_name": "example.pdf",
      "page_number": 3,
      "chunk_index": 12,
      "score": 0.82,
      "text": "Contenu du passage correspondant..."
    }
  ]
}
```

The API does not need to generate answers using an LLM. Returning the most relevant chunks and their metadata is sufficient for this exercise.

The API design, request/response schema and project structure are left to your discretion. We are interested in understanding how you design and organise the service, as well as the trade-offs behind your decisions.

3. Dockerisation

Provide a simple Dockerised setup allowing us to run both the ingestion program and the API server locally in a reproducible way. A single Dockerfile is sufficient; Docker Compose is optional.

The solution should make it clear how the generated index and metadata are persisted between the ingestion step and the API server. For example, you may use a local *data/* or *storage/* folder mounted as a Docker volume.

The README should explain how to:

- build the Docker image
- download or place the PDF files locally

- run the ingestion command with the local PDF folder path
- verify that the vector index and metadata have been created
- start the FastAPI server
- call the `/search` endpoint with an example request
- rebuild the index if the PDF folder changes

We should be able to evaluate the project locally using Docker with a small number of commands. Assuming Docker is already installed, the setup should not require paid external APIs or manual configuration beyond placing the PDF files in a local folder.

Technical constraints

- Language: Python
- API framework: FastAPI
- The ingestion program must accept the PDF folder path as a command-line argument.
- Embeddings must be generated using a local open-source model runnable on CPU.
- The vector store must be runnable locally and not require a managed cloud service.
- The solution may download open-source model weights during setup or first run, but should not require paid APIs.
- The solution must be Dockerised and runnable locally.

Solution Review

Please include a short section in your README covering the following points:

- The main limitations of your solution.
- Situations where the search quality may be poor.
- Assumptions you made while designing the system.
- What you would improve if you had additional time.
- Additional components or architectural changes you would consider for a production-ready version of the system.

There are no right or wrong answers. We are interested in understanding your reasoning, your awareness of trade-offs and your ability to critically evaluate your own work.

Submission

Please submit your solution as a GitHub repository.

The repository should contain:

- The complete source code.
- Docker configuration.
- A README with setup and execution instructions.
- Any assumptions, trade-offs or design decisions you would like to highlight.

If there are areas you would improve with additional time, feel free to document them in the README.

Once your submission is ready, please send the GitHub repository link by email to **januka@datapolitics.fr**.

Please ensure that the repository remains accessible until the recruitment process is complete.

We look forward to discussing your solution, your technical choices and your approach during the interview process.

Thank you for taking the time to complete this exercise. We appreciate the effort involved and your interest in joining the Datapolitics team.